

# HTML & CSS

Defi 10 Jours - Guide de reference complet

| Jours 1-3 | Fondations solides (HTML semantique, Box model, Variables CSS) |
|-----------|----------------------------------------------------------------|
| Jours 4-5 | Mise en page moderne (Flexbox + CSS Grid)                      |
| Jours 6-7 | Responsive Design et Animations CSS                            |
| Jours 8-9 | Selecteurs avances et Architecture BEM                         |
| Jour 10   | Projet final - Portfolio professionnel                         |

# Jour 1 - Semantique HTML5 et Accessibilite

---

## Elements de structure HTML5

- En-tete de page ou de section. Peut apparaitre plusieurs fois.
- Navigation principale. aria-label si plusieurs nav sur la page.
- Contenu principal unique. Un seul par document.
- Contenu autonome. Test : a-t-il du sens dans un flux RSS ?
- Section thematique. Doit avoir un heading h2/h3 a l'interieur.
- Contenu complementaire. Sidebar, callout, note.
- Pied de page ou de section.
- Date/heure. Attribut datetime obligatoire en format YYYY-MM-DD.
- Contenu illustratif avec figcaption.
- Coordonnees de contact - PAS le nom d'un auteur.

## Attributs ARIA essentiels

- aria-label - Nom accessible quand il n'y a pas de texte visible.
- aria-labelledby - Pointe vers l'ID d'un element qui sert de label.
- aria-describedby - Pointe vers un texte d'aide ou message d'erreur.
- aria-hidden - Masque un element aux lecteurs d'ecran.
- aria-current - Indique l'element actif : page, step, date...
- aria-required - Marque un champ comme obligatoire pour les lecteurs d'ecran.
- aria-live - polite = annonce douce | assertive = interrompt

## Regle fondamentale ARIA

Utilise d'abord le bon element HTML natif. ARIA complete, il ne remplace pas.

## Formulaire accessible - structure de reference

```
<form method="post" novalidate>
  <p>Champs marques * sont requis.</p>
  <label for="email">
    Email <span aria-hidden="true">*</span>
  </label>
  <input type="email" id="email" name="email"
    required aria-required="true"
    aria-describedby="email-error"
    autocomplete="email">
  <span id="email-error" role="alert" aria-live="polite"></span>

  <fieldset>
    <legend>Sujet</legend>
    <input type="radio" id="q" name="sujet" value="question">
    <label for="q">Question</label>
  </fieldset>
  <button type="submit">Envoyer</button>
</form>
```

*Regles cles :*

OK Chaque input a un label lie par for/id - jamais de placeholder seul

OK Radios/checkboxes groupes dans fieldset + legend

OK Erreurs liees par aria-describedby avec role='alert'

OK aria-label ET label ne coexistent pas - conflit

OK button sans type='submit' dans un form = soumet le formulaire

### **article vs section - la distinction exacte**

article = contenu autonome qui a du sens sorti de son contexte (test RSS). section = groupe thematique qui necessite un titre (h2/h3). Sans titre -> utilise div.

# Jour 2 - Box Model et Positionnement

## box-sizing : la regle la plus importante

```
/* TOUJOURS commencer par ce reset */
*, *::before, *::after {
  box-sizing: border-box;
}

/* content-box (defaut) : width = contenu seul
   width:200px + padding:20px + border:2px = 244px reels

   border-box : width = tout inclus
   width:200px = taille finale garantie */
```

## Les 5 valeurs de position

| Valeur   | Comportement                      | Parent de reference              |
|----------|-----------------------------------|----------------------------------|
| static   | Flux normal - default             | --                               |
| relative | Dans le flux, decale visuellement | Sa position naturelle            |
| absolute | Sorti du flux                     | Parent positionne le plus proche |
| fixed    | Sorti du flux                     | Le viewport                      |
| sticky   | Relatif puis bloque au scroll     | Son conteneur parent             |

## Stacking context - pieges du z-index

- ! z-index ne fonctionne que sur les elements positionnes (position != static)
- ! opacity < 1, transform, filter creent un stacking context
- ! isolation: isolate cree un contexte propre sans effet visuel
- ! overflow: hidden coupe le contenu - independant du z-index
- ! Un parent avec z-index faible plafonne tous ses enfants

## Margin collapse

Deux marges verticales adjacentes fusionnent - CSS garde la plus grande, pas la somme. 40px + 40px = 40px (pas 80px). Solution : une seule marge directionnelle (toujours margin-bottom). Ou utiliser gap avec Flexbox/Grid.

## BFC - Block Formatting Context

overflow: hidden sur le parent cree un BFC : il contient ses enfants flottants et calcule sa hauteur avec eux. Aussi : display:flow-root (propre, sans effets de bord).

## Ellipsis - texte tronque

```
/* 1 ligne - 3 proprietes obligatoires */
.titre {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
```

```
}

/* Multi-lignes (webkit - standard de facto) */
.description {
  overflow: hidden;
  display: -webkit-box;
  -webkit-line-clamp: 2;
  -webkit-box-orient: vertical;
}
```

# Jour 3 - Typographie, Couleurs et Variables CSS

## Custom Properties - deux niveaux d'indirection

```
:root {
  /* Niveau 1 - palette brute */
  --color-blue-50: #eff6ff;
  --color-blue-600: #2563eb;
  --color-blue-900: #1e3a8a;
  --color-gray-50: #f9fafb;
  --color-gray-900: #111827;

  /* Niveau 2 - tokens sémantiques */
  --surface-page: var(--color-gray-50);
  --text-primary: var(--color-gray-900);
  --action-bg: var(--color-blue-600);
  --action-hover: var(--color-blue-700);

  /* Espacement systématique */
  --space-unit: 4px;
  --space-1: calc(var(--space-unit) * 1); /* 4px */
  --space-2: calc(var(--space-unit) * 2); /* 8px */
  --space-4: calc(var(--space-unit) * 4); /* 16px */
  --space-8: calc(var(--space-unit) * 8); /* 32px */
  --space-16: calc(var(--space-unit) * 16); /* 64px */

  /* Rayons */
  --radius-sm: 4px;
  --radius-md: 8px;
  --radius-lg: 12px;
  --radius-full: 9999px;
}
```

*Règle d'or : ne jamais coder une couleur en dur dans un composant. Toujours passer par les tokens.*

## clamp() - typographie fluide sans media queries

```
/* clamp(MIN, VALEUR-FLUIDE, MAX) */
:root {
  font-size: clamp(1rem, 0.5vw + 0.875rem, 1.125rem);
}

h1 { font-size: clamp(1.75rem, 4vw + 0.5rem, 2.5rem); }
h2 { font-size: clamp(1.25rem, 3vw + 0.5rem, 1.875rem); }
h3 { font-size: clamp(1.1rem, 2vw + 0.5rem, 1.5rem); }

/* Calcul du milieu : pour aller de 28px à 40px
   entre 320px et 1280px de viewport -> 4vw + 0.5rem */
```

## Unités relatives

| Unité | Relatif à                  | Usage                               |
|-------|----------------------------|-------------------------------------|
| rem   | font-size de html          | Typographie et espacements globaux  |
| em    | font-size du parent direct | Espacements internes d'un composant |

|       |                             |                                                |
|-------|-----------------------------|------------------------------------------------|
| vw/vh | Largeur/hauteur du viewport | Typographie fluide avec clamp()                |
| %     | Propriete du parent         | Largeurs fluides (padding vertical -> width !) |

### **Piege : variables CSS dans les media queries**

Les variables CSS ne fonctionnent PAS dans les conditions des media queries - seulement dans les declarations. Utiliser des valeurs literales dans @media.

# Jour 4 - Flexbox en profondeur

---

## Modele mental - axes

justify-content agit sur l'AXE PRINCIPAL. align-items agit sur l'AXE CROISE. flex-direction determine lequel est lequel (row = horizontal, column = vertical).

## Proprietes du conteneur

```
.conteneur {
  display: flex;
  flex-direction: row | row-reverse | column | column-reverse;
  flex-wrap: nowrap | wrap | wrap-reverse;
  justify-content: flex-start | center | flex-end |
    space-between | space-around | space-evenly;
  align-items: stretch | flex-start | center | flex-end | baseline;
  align-content: /* aligne les lignes en mode wrap */
  gap: 16px;      /* entre tous les items */
  gap: 16px 8px;  /* row-gap column-gap */
}
```

## Proprietes des items

```
.item {
  flex: 1;      /* = flex-grow:1 flex-shrink:1 flex-basis:0% */
  flex: auto;   /* = 1 1 auto - base sur la taille naturelle */
  flex: none;   /* = 0 0 auto - taille fixe */
  flex: 0 0 200px; /* largeur fixe 200px */

  align-self: center; /* surcharge align-items pour cet item */
  order: -1;         /* passe en premier visuellement */
  margin-left: auto; /* pousse tout a droite */
}
```

## Cas d'usage classiques

```
/* Footer colle en bas */
body { display: flex; flex-direction: column; min-height: 100vh; }
footer { margin-top: auto; }

/* Centrage parfait */
.container { display: flex; align-items: center; justify-content: center; }

/* Navbar 3 zones */
.nav { display: flex; justify-content: space-between; align-items: center; }

/* Cards hauteur egale - GRATUIT en Flexbox */
.cards { display: flex; align-items: stretch; /* default */ }

/* Item qui prend tout l'espace disponible */
.main { flex: 1; }
.sidebar { flex: 0 0 280px; }
```



# Jour 5 - CSS Grid en profondeur

---

## Unites et fonctions Grid

```
.grid {
  display: grid;

  /* fr = fraction de l'espace disponible */
  grid-template-columns: 1fr 2fr 1fr; /* 3 colonnes */
  grid-template-columns: repeat(3, 1fr);

  /* minmax(min, max) */
  grid-template-columns: minmax(200px, 1fr) 3fr;

  /* Responsive SANS media query */
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));

  /* auto-fit vs auto-fill :
    auto-fit : efface les colonnes vides, etire les items
    auto-fill : conserve les colonnes vides */

  gap: 1.5rem;
  grid-auto-rows: minmax(100px, auto);
}
```

## grid-template-areas

```
.layout {
  display: grid;
  grid-template-columns: 260px 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header"
    "sidebar main "
    "footer footer";
  min-height: 100vh;
}

header { grid-area: header; }
.sidebar { grid-area: sidebar; }
main { grid-area: main; }
footer { grid-area: footer; }

/* Mobile - une seule media query */
@media (max-width: 768px) {
  .layout {
    grid-template-columns: 1fr;
    grid-template-areas:
      "header"
      "main"
      "sidebar"
      "footer";
  }
}
```

## Placement explicite

```
.item {
  grid-column: 1 / 3;      /* de la ligne 1 a la ligne 3 */
  grid-column: span 2;     /* occupe 2 colonnes */
  grid-column: 1 / -1;    /* toute la largeur */
  grid-row: span 2;       /* occupe 2 rangees */
  grid-row: 2 / 4;        /* de la rangee 2 a 4 */
}
```

```
/* Le . dans les areas = cellule vide */
```

```
grid-template-areas:
```

```
  "header header"
```

```
  "nav    main  "
```

```
  ".      main  "
```

```
  "footer footer";
```

# Jour 6 - Responsive Design et Container Queries

## Mobile-first - la bonne approche

```
/* MOBILE-FIRST : on enrichit progressivement */
.grid { grid-template-columns: 1fr; }
@media (min-width: 768px) { .grid { grid-template-columns: repeat(2, 1fr); } }
@media (min-width: 1024px) { .grid { grid-template-columns: repeat(3, 1fr); } }

/* Desktop-first (à éviter) : on défait */
.grid { grid-template-columns: repeat(3, 1fr); }
@media (max-width: 768px) { .grid { grid-template-columns: 1fr; } }
```

## Breakpoints standards

| Nom | min-width | Cible          |
|-----|-----------|----------------|
| sm  | 480px     | Petits mobiles |
| md  | 768px     | Tablettes      |
| lg  | 1024px    | Laptop         |
| xl  | 1280px    | Desktop        |
| 2xl | 1536px    | Large desktop  |

## Container Queries - revolution

Repondent à la taille du CONTENEUR PARENT, pas du viewport. Un composant peut s'adapter à son espace disponible peu importe où il est placé.

```
/* 1 - déclarer un conteneur */
.card-wrapper {
  container-type: inline-size;
  container-name: card;
}

/* 2 - interroger depuis l'enfant */
@container card (width >= 400px) {
  .card {
    display: grid;
    grid-template-columns: 120px 1fr;
  }
}
```

## Images responsives

```


/* CSS */
.card-img { width: 100%; height: 200px; object-fit: cover; object-position: center; }
img { max-width: 100%; height: auto; } /* reset de base */
```

# Jour 7 - Animations et Transitions CSS

## Regles d'or des transitions

OK Toujours declarer transition sur l'etat de REPOS - pas dans :hover

OK Ne jamais utiliser transition: all - lister les proprietes explicitement

OK Animer uniquement transform et opacity pour les perfs GPU

OK Inclure prefers-reduced-motion a la fin du fichier CSS

```
.btn {
  background: #2563eb;
  transform: translateY(0);
  /* Transition sur l'etat de REPOS */
  transition: transform 0.15s ease, box-shadow 0.15s ease,
             background-color 0.2s ease;
}
.btn:hover      { transform: translateY(-2px); box-shadow: 0 4px 12px rgba(0,0,0,.15); }
.btn:active     { transform: translateY(0); box-shadow: none; }
.btn:focus-visible { outline: none; box-shadow: 0 0 3px rgba(37,99,235,.4); }
.btn:disabled   { opacity: 0.4; cursor: not-allowed; }
```

## Performance - proprietes GPU vs CPU

| Propriete              | Phase           | Recommandation                         |
|------------------------|-----------------|----------------------------------------|
| transform, opacity     | Composite (GPU) | OK - a utiliser pour animer            |
| color, background      | Paint           | OK avec moderation                     |
| width, height, padding | Layout          | EVITER - recalcul de toute la page     |
| margin, top, left      | Layout          | EVITER - utiliser transform a la place |

## Keyframes - animations

```
@keyframes fade-in {
  from { opacity: 0; transform: translateY(12px); }
  to   { opacity: 1; transform: translateY(0); }
}
.element { animation: fade-in 0.6s ease forwards; }

@keyframes pulse {
  0%, 100% { transform: scale(1); }
  50%      { transform: scale(1.1); }
}
.badge { animation: pulse 2s ease-in-out infinite; }

@keyframes spin {
  to { transform: rotate(360deg); }
}
.spinner { animation: spin 0.8s linear infinite; }
```

## prefers-reduced-motion - obligatoire

```
/* A la fin de CHAQUE fichier CSS avec des animations */
@media (prefers-reduced-motion: reduce) {
```

```
*, *::before, *::after {  
  transition: none !important;  
  animation: none !important;  
}  
}
```

# Jour 8 - Selecteurs avances et CSS moderne

## Pseudo-classes structurales

```
li:nth-child(odd)    { background: #f5f5f5; } /* lignes impaires */
li:nth-child(2n)     { }                    /* lignes paires */
li:nth-child(3n+1)   { }                    /* 1er de chaque groupe de 3 */
li:nth-child(n+4)     { display: none; }      /* a partir du 4e */
li:not(:last-child)  { border-bottom: 1px solid #eee; } /* tous sauf le dernier */
:first-child, :last-child, :only-child
:empty               /* elements sans enfants */
:checked, :required, :valid, :invalid /* etats de formulaire */
:focus-visible       /* focus clavier uniquement */
```

## :is(), :where() et :has()

```
/* :is() - groupe, specificite du + specifique */
:is(header, main) :is(h1, h2, h3) { color: var(--text-primary); }

/* :where() - identique mais specificite ZERO - facilement surchargeable */
:where(h1, h2, h3) { line-height: 1.2; font-weight: 600; }

/* :has() - selecteur parent (disponible depuis 2023) */
.card:has(img)           { grid-template-columns: 120px 1fr; }
label:has(+ input:invalid) { color: red; }
figure:not(:has(figcaption)) { padding-bottom: 0; }
nav:has(a[aria-current="page"]) { background: var(--bg-accent); }
form:has(input:invalid:not(:placeholder-shown)) .submit { opacity: 0.5; }
```

## Pseudo-elements ::before et ::after

```
/* Compteur CSS */
ol { counter-reset: items; }
li { counter-increment: items; }
li::before { content: counter(items); /* cercle numerote */ }

/* Badge requis */
label.required::after { content: ""; color: red; margin-left: 3px; }

/* Icone automatique sur les liens */
a[href^="http"]::after { content: " ->" ; font-size: 11px; }
a[href^="mailto"]::before { content: "@ "; }

/* Separateur avec lignes */
.divider { position: relative; text-align: center; }
.divider::before, .divider::after {
  content: "";
  position: absolute; top: 50%; height: 1px;
  background: var(--border); width: calc(50% - 60px);
}
.divider::before { left: 0; }
.divider::after { right: 0; }
```

## Composants CSS purs - pattern checkbox

```
/* Input cache + Label style + :checked pour reagir */
.toggle-input { position: absolute; opacity: 0; }
```

```
.toggle-input:checked + .toggle-label { background: var(--accent); }
.toggle-input:checked + .toggle-label::after { transform: translateX(20px); }

/* Accordion */
.acc-body { max-height: 0; overflow: hidden; transition: max-height .3s ease; }
.acc-input:checked ~ .acc-body { max-height: 200px; }

/* Selecteur ~ = frere general | + = frere adjacent */
```

# Jour 9 - Architecture CSS et Methodologie BEM

## Convention BEM - Block\_\_Element--Modifier

Le double separateur EST la convention. card\_title n'est pas du BEM. card\_\_title oui. La difference visuelle permet de lire instantanement block, element et modifier.

```
/* Block - composant independant */
.card { border-radius: 12px; overflow: hidden; }

/* Elements - parties du block (DOUBLE underscore) */
.card__header { position: relative; }
.card__image { width: 100%; height: 200px; object-fit: cover; }
.card__body { padding: 1rem; }
.card__title { font-size: 1.1rem; font-weight: 600; }
.card__text { font-size: 13px; color: #6b7280; }
.card__footer { display: flex; justify-content: space-between; }

/* Modifiers - variantes (DOUBLE tiret) */
.card--featured { border: 2px solid var(--accent); }
.card--horizontal { display: flex; width: 400px; }
.card__title--large { font-size: 1.4rem; }
.card__image--small { width: 120px; height: 100%; }
```

## 4 regles BEM a respecter

OK Un element appartient a UN seul block

OK Modifier = s'ajoute a la base : class='btn btn--primary' (jamais seul)

OK Pas d'imbrication dans le nom : .card\_\_title pas .card\_\_header\_\_title

OK Specificite max (0,1,0) - une seule classe, zero imbrication CSS

## Erreurs frequentes

X Underscore simple card\_title au lieu de double underscore card\_\_title

X Modifier sans base : class='btn--primary' au lieu de class='btn btn--primary'

X Imbrication CSS : .card .card\_\_title { } au lieu de .card\_\_title { }

X Melange de blocks : class='btn card\_\_action' - choisir l'un ou l'autre

## Architecture des fichiers

```
styles/
tokens/
  colors.css          <- rampes 50->900
  typography.css      <- echelle typo clamp()
  spacing.css         <- --space-unit et derivees
base/
  reset.css           <- box-sizing, margins
  globals.css         <- html, body, liens
components/
  button.css          <- .btn, .btn--*
  card.css            <- .card, .card__*, .card--*
  nav.css             <- .nav, .nav__*
  form.css            <- .form, .form__*
layouts/
  grid.css            <- .layout, .grid
```



page.css  
main.css

```
<- .page, .sidebar  
<- @import dans l'ordre
```

# Checklist Finale - Avant chaque livraison

---

## HTML

- ☐ Un seul h1 par page - hierarchie h1 a h6 sans saut de niveau
- ☐ Tous les inputs ont un label lie par for/id
- ☐ Toutes les images ont un alt descriptif (vide alt="" si decorative)
- ☐ Les dates utilisent time datetime='YYYY-MM-DD'
- ☐ aria-current='page' sur le lien actif de la navbar
- ☐ Les radios et checkboxes groupes dans fieldset + legend
- ☐ button type='submit' uniquement sur le bouton d'envoi

## CSS

- ☐ \*, ::before, ::after { box-sizing: border-box; } en tout premier
- ☐ Zero valeur magique - tout passe par les tokens :root
- ☐ BEM strict - double underscore \_\_ et double tiret --
- ☐ Mobile-first - min-width dans toutes les media queries
- ☐ prefers-reduced-motion en bas du fichier CSS
- ☐ Transitions sur l'etat de repos, proprietes explicites (pas 'all')
- ☐ focus-visible sur tous les elements interactifs
- ☐ HTML et CSS coherents - chaque classe HTML a sa regle CSS

## Performance et Accessibilite

- ☐ Animer uniquement transform et opacity
- ☐ will-change uniquement sur les elements qui vont animer
- ☐ Images avec srcset + sizes pour le responsive
- ☐ object-fit: cover sur les images en conteneur de taille fixe
- ☐ Contraste texte/fond minimum 4.5:1 (WCAG AA)
- ☐ Navigation possible au clavier - tester Tab et Shift+Tab

---

## 10 questions d'entretien technique HTML/CSS

1. Ecrire un layout complet avec grid-template-areas de memoire
2. Expliquer la difference entre position absolute et fixed
3. Corriger un z-index qui ne fonctionne pas (stacking context)
4. Expliquer le margin collapse et les 3 facons de le resoudre
5. Ecrire un clamp() pour une typographie fluide h1 mobile->desktop
6. Nommer correctement un composant en BEM (block, element, modifier)
7. Centrer un element en Flexbox et en Grid (2 facons chacun)
8. Ecrire une transition correcte avec prefers-reduced-motion
9. Expliquer la difference entre :is() et :where() (specificite)
10. Differences entre display:none, visibility:hidden et opacity:0

---

Bon courage pour tes entretiens et tes projets !

Tu as les bases solides pour construire des interfaces professionnelles.